

Reviewer Pack — Free Cumulant Gap in Read-Twice Branching Programs for IP_2

Entry e97847babd53 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 06:11:41 UTC

Free Cumulant Gap in Read-Twice Branching Programs for IP_2

Entry ID: e97847babd53 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 06:11:41 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Read-Twice Branching Programs (BP)

Statement:

For a read-twice BP P computing IP_2 , the free cumulant gap $\rho(P) = \Omega(n)$, whereas for any read-once BP Q , $\rho(Q) = O(\log n)$.

Rationale (proposer's reasoning):

Free cumulants capture the non-commutative structure of the BP's transitions, and the exponential gap in IP_2 's complexity suggests a corresponding gap in free cumulant measures. This invariant distinguishes read-twice BPs from read-once ones by leveraging the algebraic properties of non-commutative random variables.

Taxonomy category: BP_READTWICE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b8c539877cecd932

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_bp(n, read_twice=False):
        if read_twice:
            # Generate a read-twice BP for IP_2
            bp = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
            return bp
        else:
            # Generate a read-once BP
            bp = [[random.randint(0, 1) if i != j else 0 for j in range(n)] for i in range(n)]
            return bp

    def matrix_multiply(A, B):
        n = len(A)
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def gaussian_elimination(A, b):
        n = len(A)
        for i in range(n):
            max_row = i
            for j in range(i + 1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]
            for j in range(i + 1, n):
                factor = A[j][i] / A[i][i]
                A[j][i] = 0
                for k in range(i + 1, n):
                    A[j][k] -= factor * A[i][k]
                b[j] -= factor * b[i]
```

```

x = [0] * n
for i in range(n - 1, -1, -1):
    x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i + 1, n))) / A[i][i]
return x

def free_cumulant_gap(bp):
    n = len(bp)
    transition_matrix = matrix_multiply(bp, bp)
    eigenvalues = []
    for _ in range(10): # Simple power iteration to approximate eigenvalues
        v = [random.random() for _ in range(n)]
        v /= math.sqrt(sum(x**2 for x in v))
        v_next = matrix_multiply(transition_matrix, v)
        v_next /= math.sqrt(sum(x**2 for x in v_next))
        lambda_ = sum(v[i] * v_next[i] for i in range(n))
        eigenvalues.append(lambda_)
    return max(eigenvalues) - min(eigenvalues)

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    read_once_bp = generate_bp(n, read_twice=False)
    read_twice_bp = generate_bp(n, read_twice=True)

    rho_Q = free_cumulant_gap(read_once_bp)
    rho_P = free_cumulant_gap(read_twice_bp)

    results.append({
        "n": n,
        "rho_Q": rho_Q,
        "rho_P": rho_P
    })

instances_tested = len(results) * len(n_values)
conjecture_holds = all(rho_Q <= math.log(n) and rho_P >= n / 2 for r in results for n, _, _ in results)
counterexample = "" if conjecture_holds else "Read-twice BP computing IP_2 with  $\rho(P) < n/2$ "

return {
    "metric_name": "Free Cumulant Gap",
    "metric_value": sum(r["rho_P"] for r in results),
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)

```

```

std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"Read-twice BP computing IP_2 with  $p(P) < n/2$ \")")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c2fd3bf7.py", line 106, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c2fd3bf7.py", line 79, in run_trial
    rho_Q = free_cumulant_gap(read_once_bp)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c2fd3bf7.py", line 66, in free_cumulant_gap
    v /= math.sqrt(sum(x**2 for x in v))
TypeError: unsupported operand type(s) for /=: 'list' and 'float'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to TypeError, preventing data collection. Pre-registered support condition cannot be evaluated. | next: Debug the free_cumulant_gap function's division operation to handle list types properly

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	106281
2	propose	ollama_remote	qwen3:8b	0	0	101303
3	preregistration	ollama_remote	qwen3:8b	0	0	19994
4	novelty	ollama_remote	qwen3:8b	0	0	16605

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	novelty	ollama_remote	qwen3:8b	0	0	10184
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16101
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13286
8	judge	ollama_remote	qwen3:8b	0	0	9203

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 292958 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/e97847babd53.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e97847babd53.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e97847babd53.tar.gz (if generated)